Overview

Package Load

Step 1: Import Data and Fit Ising Network

Step 2: Moderation Effect Test (Prerequisite for NIRA)

Step 3: NIRA Simulation Intervention

Step 4: Permutation Test

Step 5: Stability Test

Summary: Three-Step Validation Procedure

Session Info

NIRApost Tutorial

Wang Fei, Wu Yiming, Wu Yibo, Zhu Tingshao

2025-05-25

Overview

NIRApost is an R package designed for post-processing of the NodeIdentifyR Algorithm (NIRA) applied to Ising network models in psychological research. It implements a three-step validation procedure:

1. **Moderation effect testing** (`runMgmmAnalysis`) — a theoretical prerequisite that must be completed *before* running NIRA
2. **Permutation testing** (`permutationNIRAtest` , `plotMeanNIRA`) — formal significance testing with multiple-comparison correction
3. **Stability testing** (`stabilityNIRAtest` , `findMaxN` , `plotMaxN`) — verification that results are reproducible across independent simulation runs

The package is used in conjunction with **nodeIdentifyR**. For full details, please refer to the companion tutorial article:

> *Simulation Intervention for Cross-Sectional Network Models: An R Tutorial*

Package Load

```
library(NIRApost)        # Load NIRApost Package
library(bootnet)         # Load bootnet Package
library(nodeIdentifyR)   # Load nodeIdentifyR Package
library(dplyr)           # Load dplyr Package
```

Step 1: Import Data and Fit Ising Network

We use the built-in `single_gds` dataset (GAD-7 anxiety scale, N = 2,404). An Ising network is estimated via `bootnet::estimateNetwork()` with the "AND" rule. Two components are extracted:

- **Edge weight matrix** — pairwise connection weights between all nodes
- **Threshold parameter vector** — intercept terms from each node's logistic regression in the Ising model

```
# Load the built-in GAD-7 anxiety dataset
data("single_gds")

# Fit the Ising model
gs_fit <- estimateNetwork(single_gds,
                          default = "IsingFit",
                          rule    = "AND")

# Extract edge weight matrix and threshold parameters
edgeWeightMatrix <- gs_fit$graph
thresholdVector  <- gs_fit$intercepts
```

Step 2: Moderation Effect Test (Prerequisite for NIRA)

Why This Must Be Done First

NIRA perturbs one node's threshold while keeping all edge weights unchanged. This is valid **only if** no node moderates other pairwise edges. If node A moderates the B–C relationship, then changing A's threshold implicitly alters the B–C edge weight, making the original edge weight matrix theoretically inappropriate for post-intervention simulation.

This test must be run before NIRA. An empty `significant_moderators` list confirms the assumption holds.

Implementation

`runMgmmAnalysis()` iteratively treats each node as a candidate moderator, fits a Moderated Mixed Graphical Model (MGMM) via `mgm::mgm()`, and uses resampling via `mgm::resample()` to retain only stable effects (95% CI excluding zero).

Parameter	Description
`data`	Dataset as a numeric matrix
`plotResults`	`TRUE` exports per-moderator network plots
`rule`	`"AND"` (stringent, default) or `"OR"` (inclusive)
`lambdaGam`	EBIC regularization parameter (default: 0.25)

Parameter	Description
`nB`	Resampling iterations (use ≥ 100 for publication)

```
# nB = 10 for demonstration only; use >= 100 for publication
sig_moder <- runMgmmAnalysis(
  data        = as.matrix(single_gds),
  plotResults = FALSE,
  rule        = "AND",
  lambdaGam   = 0.25,
  nB          = 10
)

# Empty list => no moderation effects => NIRA is appropriate
print(sig_moder$significant_moderators)
```

Step 3: NIRA Simulation Intervention

With the moderation assumption confirmed, we proceed with NIRA using functions from **nodeIdentifyR**.

`simulateResponses()` generates N + 1 datasets: for each of the N nodes, one dataset with that node's threshold reduced by 2 SDs (alleviating intervention), plus one unperturbed original dataset.

To perform aggravating interventions, change `"alleviating"` to `"aggravating"` throughout.

```
set.seed(2025)

# Generate N+1 simulated samples
gs_IsingSamples <- simulateResponses(
  edge_weight                = edgeWeightMatrix,
  thresholds                 = thresholdVector,
  perturbation_type          = "alleviating",
  amount_of_SDs_perturbation = 2
)

# Calculate sum scores for each participant in each simulated sample
gs_sumIsingSamples <- calculateSumScores(gs_IsingSamples)

# Convert to long format for plotting
gs_sumIsingSamplesLong <- prepareDFforPlottingAndANOVA(gs_sumIsingSamples)

# Plot mean total scores (ascending order for alleviating interventions)
plotSumScores(
  sum_scores_long   = gs_sumIsingSamplesLong,
  perturbation_type = "alleviating"
)
```

Step 4: Permutation Test

Why Visual Inspection Is Insufficient

The `plotSumScores` output has four critical limitations:

1. **No formal evidence** — visual comparison cannot confirm statistical significance
2. **CI overlap is unreliable** — checking 95% error bar overlap is an inaccurate significance criterion
3. **Magnitude sensitivity** — smaller perturbations make visual assessment highly unreliable
4. **Multiple-comparison inflation** — N simultaneous comparisons with no correction inflate the Type I error rate

Implementation

`permutationNIRAtest()` calls `permutationTest()` for every node in batch, applies multiple-comparison correction, and returns a unified statistics table and plotting-ready data frame. `plotMeanNIRA()` visualises the result as a line plot with 95% CIs and significance stars (*** p < 0.001, ** p < 0.01, * p < 0.05).

Parameter	Description
`gs_sumIsingSamplesLong`	Long-format output from `prepareDFforPlottingAndANOVA()`
`method`	Multiple-comparison correction: `"bonferroni"` (default), `"BH"`, `"holm"`, etc.

```
# Batch permutation tests with Bonferroni correction
stat_result <- permutationNIRAtest(gs_sumIsingSamplesLong,
                                   method = "bonferroni")

# View per-node statistics (mean, SD, SE, CI, Cohen's d, p, p.adjust)
head(stat_result$stat)

# Visualise means with 95% CIs and significance stars
plotMeanNIRA(stat_result, "alleviating")
```

Step 5: Stability Test

Why Stability Testing Is Necessary

A single simulation run may give different results depending on the random seed. Only results consistent across many independent simulation runs can be trusted.

Note: this procedure performs *repeated simulation* — independently re-running the full simulation from scratch using the original network parameters. The observed data are never resampled.

Implementation

`stabilityNIRAtest()` repeats the full simulation `nReps` times independently. `findMaxN()` aggregates how often each node ranked in the top-n positions across all repetitions. `plotMaxN()` visualises the stability as a stacked bar plot.

Parameter	Description
`edge_weights`	Edge weight matrix from the Ising network
`thresholds`	Threshold parameter vector
`perturbation_type`	`"aggravating"` or `"alleviating"`
`amount_of_SDs_perturbation`	Perturbation magnitude in SDs
`nReps`	Number of independent simulation repetitions
`parallel`	`TRUE` enables parallel execution (recommended for nReps ≥ 100)
`ncores`	CPU cores (`NULL` = auto-detect)

```
# Parallel execution (recommended)
# nReps = 100 for demonstration; use >= 1000 for publication
simResult <- stabilityNIRAtest(
  edge_weights               = edgeWeightMatrix,
  thresholds                 = thresholdVector,
  perturbation_type          = "alleviating",
  amount_of_SDs_perturbation = 2,
  nReps                      = 100,
  parallel                   = TRUE,
  ncores                     = 6
)

# Serial execution (uncomment to use; displays a progress bar)
# simResult <- stabilityNIRAtest(
#   edgeWeightMatrix, thresholdVector,
#   "alleviating", 2,
#   nReps    = 100,
#   parallel = FALSE,
#   ncores   = NULL
# )

# Compute rank frequencies across all simulation repetitions
repeat_node <- findMaxN(simResult, n = 7)
print(repeat_node)

# Stacked bar plot of node ranking stability
plotMaxN(repeat_node, low = 1, high = 7)
```

Summary: Three-Step Validation Procedure

Step	Purpose	Primary function(s)	Pass criterion
Step 2 Moderation test	Verify NIRA assumption	`runMgmmAnalysis()`	`significant_moderators` is empty
Step 4 Permutation test	Formal significance + correction	`permutationNIRAtest()`, `plotMeanNIRA()`	Adjusted $p < 0.05$
Step 5 Stability test	Reproducibility across seeds	`stabilityNIRAtest()`, `findMaxN()`, `plotMaxN()`	Consistent top node

Session Info

```
## R version 4.4.1 (2024-06-14 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 22631)
##
## Matrix products: default
##
##
## locale:
## [1] LC_COLLATE=Chinese (Simplified)_China.utf8
## [2] LC_CTYPE=Chinese (Simplified)_China.utf8
## [3] LC_MONETARY=Chinese (Simplified)_China.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=Chinese (Simplified)_China.utf8
##
## time zone: Asia/Shanghai
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] NIRApost_1.1.0      modnets_0.9.0       nodeIdentifyR_1.0.0 dplyr_1.1.4
##  [5] qgraph_1.9.8        bootnet_1.6         ggplot2_4.0.1       testthat_3.2.3
##  [9] readxl_1.4.5        roxygen2_7.3.2      devtools_2.4.5      usethis_3.1.0
##
## loaded via a namespace (and not attached):
##   [1] splines_4.4.1        later_1.4.3          R.oo_1.27.1
##   [4] tibble_3.2.1         cellranger_1.1.0     pagedown_0.23
##   [7] rpart_4.1.23         lifecycle_1.0.4      Rdpack_2.6.4
##  [10] doParallel_1.0.17    rprojroot_2.1.0      processx_3.8.6
##  [13] lattice_0.22-6       MASS_7.3-65          backports_1.5.0
##  [16] magrittr_2.0.3       vcd_1.4-13           Hmisc_5.1-3
##  [19] sass_0.4.10          rmarkdown_2.29       plotrix_3.8-4
##  [22] jquerylib_0.1.4      yaml_2.3.10          remotes_2.5.0
##  [25] IsingFit_0.4         httpuv_1.6.16        doRNG_1.8.6.2
##  [28] sp_2.2-0             sessioninfo_1.2.3    pkgbuild_1.4.8
##  [31] pbapply_1.7-4        minqa_1.2.8          RColorBrewer_1.1-3
##  [34] abind_1.4-8          pkgload_1.4.0        quadprog_1.5-8
##  [37] R.utils_2.13.0       purrr_1.1.0          itertools_0.1-3
##  [40] nnet_7.3-19          gdata_3.0.1          ellipse_0.5.0
##  [43] mgm_1.2-15           arm_1.14-4           missForest_1.5
##  [46] codetools_0.2-20     xml2_1.4.0           tidyselect_1.2.1
##  [49] shape_1.4.6.1        farver_2.1.2         IsingSampler_0.2.4
##  [52] lme4_1.1-37          stats4_4.4.1         base64enc_0.1-3
##  [55] eigenmodel_1.11      jsonlite_2.0.0       e1071_1.7-16
##  [58] mitml_0.4-5          ellipsis_0.3.2       Formula_1.2-5
##  [61] survival_3.8-3       iterators_1.0.14     foreach_1.5.2
##  [64] progress_1.2.3       tools_4.4.1          snow_0.4-4
##  [67] NetworkToolbox_1.4.4 Rcpp_1.0.12          glue_1.7.0
##  [70] mnormt_2.1.1         gridExtra_2.3        pan_1.9
```

```
##  [73] laeken_0.5.3         xfun_0.53            ranger_0.17.0
##  [76] websocket_1.4.4      withr_3.0.2          fastmap_1.2.0
##  [79] boot_1.3-31          xopen_1.0.1          VIM_6.2.2
##  [82] callr_3.7.6          digest_0.6.36        rcmdcheck_1.4.0
##  [85] R6_2.6.1             mime_0.13            mice_3.18.0
##  [88] colorspace_2.1-0     gtools_3.9.5         weights_1.1.2
##  [91] jpeg_0.1-10          R.methodsS3_1.8.2    tidyr_1.3.1
##  [94] generics_0.1.4       ggsci_3.2.0          data.table_1.15.4
##  [97] corpcor_1.6.10       robustbase_0.99-4-1  class_7.3-23
## [100] prettyunits_1.2.0    htmlwidgets_1.6.4    pkgconfig_2.0.3
## [103] gtable_0.3.6         lmtest_0.9-40        S7_0.2.0
## [106] brio_1.1.5           htmltools_0.5.8.1    lavaan_0.6-19
## [109] carData_3.0-5        profvis_0.4.0        scales_1.4.0
## [112] Rmisc_1.5.1          png_0.1-8            wordcloud_2.6
## [115] reformulas_0.4.1     servr_0.32           knitr_1.50
## [118] rstudioapi_0.17.1    reshape2_1.4.4       coda_0.19-4.1
## [121] checkmate_2.3.1      nlme_3.1-164         curl_7.0.0
## [124] nloptr_2.2.1         proxy_0.4-27         cachem_1.1.0
## [127] zoo_1.8-14           stringr_1.5.1        parallel_4.4.1
## [130] miniUI_0.1.2         foreign_0.8-86       desc_1.4.3
## [133] pillar_1.11.0        grid_4.4.1           vctrs_0.6.5
## [136] urlchecker_1.0.1     promises_1.3.3       randomForest_4.7-1.2
## [139] car_3.1-3            jomo_2.7-6           xtable_1.8-4
## [142] cluster_2.1.6        waldo_0.6.2          htmlTable_2.4.3
## [145] evaluate_1.0.4       pbivnorm_0.6.0       tinytex_0.58
## [148] mvtnorm_1.3-3        cli_3.6.3            compiler_4.4.1
## [151] smacof_2.1-7         crayon_1.5.3         rlang_1.1.4
## [154] rngtools_1.5.2       labeling_0.4.3       fdrtool_1.2.17
## [157] ps_1.9.1             plyr_1.8.9           fs_1.6.6
## [160] stringi_1.8.4        psych_2.5.6          nnls_1.6
## [163] networktools_1.6.0   interactionTest_1.2  glmnet_4.1-10
## [166] Matrix_1.7-0         hms_1.1.3            glasso_1.11
## [169] shiny_1.11.1         rbibutils_2.3        igraph_2.0.3
## [172] broom_1.0.9          memoise_2.0.1        bslib_0.9.0
## [175] DEoptimR_1.1-4       polynom_1.4-1
```